CurrentDataService хранит последние оперативные данные в памяти запущенного RGSCenter. Это Java-сервис, а не таблица БД и не отдельный REST endpoint.

1. Структура витрины

Map<String, Map<String, Object>>

код устройства → параметр → значение

Например:

"scales"

├── pinged = true

├── interaction_lastdate_in = 1789120800

├── interaction_lastdate_out = 1789120801

├── data_available = true

├── last_data = ScalesReading(...)

└── last_error = ""

"RoboticManipulator"

├── pinged = true

├── x = 125.4

├── y = 310.8

├── z = 84.2

├── digital_inputs = 5

├── robot_mode = 5

└── current_command_id = 102

2. Стандартные данные всех устройств

Ключ	Java-тип	Значение
pinged	Boolean	Последняя проверка связи успешна
interaction_lastdate_in	Long	Epoch time в секундах отправки последнего запроса

Ключ	Java-тип	Значение
interaction_lastdate_out	Long	Epoch time в секундах получения последнего ответа
data_available	Boolean	Есть ли актуальный успешный результат
last_data	Object	Последние полученные прикладные данные
last_error	String	Текст последней ошибки

Начальные значения после запуска:

pinged = false

interaction_lastdate_in = 0

interaction_lastdate_out = 0

data_available = false

last_data = ""

last_error = ""

3. Что находится в last_data

Тип значения зависит от устройства:

Устройство	Содержимое last_data
Nicole	NicoleSnapshot
SHP	ShpMeasurements
Весы	ScalesReading
Gamma-PRO	GammaProSnapshot
BOSON	BosonHighVoltageSnapshot
Сканер	String со штрих-кодом
Манипулятор	DobotFeedbackFrame
GDRM	GdrmMeasurement

Camera, LenovoPC преимущественно общие результаты проверки связи

Снимок камеры и результат распознавания бутылок находятся отдельно в CameraImagingService, а не в CurrentDataService.

4. Специальные параметры манипулятора

Сейчас RoboticManipulatorStreamService дополнительно публикует:

Ключ	Тип
x, y, z	Double
rx, ry, rz	Double
joint_1 ... joint_N	Double
digital_inputs	Long
digital_outputs	Long
robot_mode	Long
queue_running	Boolean
enabled	Boolean
running	Boolean
error_status	Boolean
current_command_id	Long
safety_state	Long
telemetry_stale	Boolean
di_0, di_1	Boolean
spectrometer_hatch_closed	Boolean
spectrometer_hatch_open	Boolean
hatch_analog_input_index	Integer
hatch_analog_input	Double

Номера di_0 и di_1 зависят от настроенных битов концевиков.

5. Чтение одного параметра из Java

Сначала внедряется Spring-сервис:

```
@Service
public class ExampleService {

  private final CurrentDataService currentDataService;

  public ExampleService(CurrentDataService currentDataService) {
    this.currentDataService = currentDataService;
  }
}
```

Проверка доступности весов:

```
boolean online = currentDataService.getValue(
    DeviceCode.SCALES,
    CurrentDataService.PINGED,
    Boolean.class
);
```

Получение времени последнего ответа:

```
long lastResponse = currentDataService.getValue(
    DeviceCode.SCALES,
    CurrentDataService.INTERACTION_LASTDATE_OUT,
    Long.class
);
```

Преобразование epoch time:

```
Instant responseTime = Instant.ofEpochSecond(lastResponse);
```

6. Чтение последних данных

Перед чтением last_data обязательно проверяется data_available:

```
boolean available = currentDataService.getValue(
```

```
    DeviceCode.SCALES,
    CurrentDataService.DATA_AVAILABLE,
    Boolean.class
);

if (available) {
  ScalesReading reading = currentDataService.getValue(
      DeviceCode.SCALES,
      CurrentDataService.LAST_DATA,
      ScalesReading.class
  );

  BigDecimal grams = reading.massGrams();
}
```

Без проверки data_available поле last_data может содержать начальную пустую строку или устаревший результат.

7. Чтение параметров манипулятора

```
double x = currentDataService.getValue(
    DeviceCode.ROBOTIC_MANIPULATOR,
    "x",
    Double.class
);

long digitalInputs = currentDataService.getValue(
    DeviceCode.ROBOTIC_MANIPULATOR,
    "digital_inputs",
    Long.class
```

```
);


boolean hatchClosed = currentDataService.getValue(
    DeviceCode.ROBOTIC_MANIPULATOR,
    "spectrometer_hatch_closed",
    Boolean.class
);
```

Но до получения первой телеметрии специальных ключей может ещё не быть. Тогда getValue() выбросит NoSuchElementException.

Безопасный вариант:

```
Map<String, Object> data = currentDataService.getDeviceData(
    DeviceCode.ROBOTIC_MANIPULATOR
);


Double x = data.get("x") instanceof Double value ? value : null;
Boolean hatchClosed =
    data.get("spectrometer_hatch_closed") instanceof Boolean value
        ? value
        : null;
```

8. Получение всей записи устройства

```
Map<String, Object> scalesData =
    currentDataService.getDeviceData(DeviceCode.SCALES);
```

Возвращается неизменяемая копия:

```
Object pinged = scalesData.get("pinged");
Object lastData = scalesData.get("last_data");
Object error = scalesData.get("last_error");
```

9. Получение всей витрины

```
Map<String, Map<String, Object>> allData =
    currentDataService.snapshot();
```

Перебор:

```
for (Map.Entry<String, Map<String, Object>> device : allData.entrySet()) {
  String deviceCode = device.getKey();
  Map<String, Object> parameters = device.getValue();

  System.out.println(deviceCode + ": " + parameters);
}
```

Снимок будет содержать разделы:

camera

lenovo-pc

nicole

boson

RoboticManipulator

barcode-scanner

gdrm

scales

shp

gamma-pro

10. Запись значений

Записывать данные должны сервисы устройств, а не web-контроллеры.

Одно поле:

```
currentDataService.setValue(
    DeviceCode.SCALES,
    CurrentDataService.PINGED,
    true
```

);

Несколько согласованных полей:

```
Map<String, Object> values = new LinkedHashMap<>();
values.put("x", 120.5);
values.put("y", 300.0);
values.put("robot_mode", 5L);
values.put("running", false);

currentDataService.setValues(
    DeviceCode.ROBOTIC_MANIPULATOR,
    values
);
```

Успешное получение прикладных данных:

```
currentDataService.recordDataSuccess(
    DeviceCode.SCALES,
    reading
);
```

При этом устанавливается:

```
last_data            = reading
data_available       = true
last_error           = ""
interaction_lastdate_out = текущее время
```

Ошибка:

```
currentDataService.recordDataFailure(
    DeviceCode.SCALES,
    "Scales response timed out"
);
```

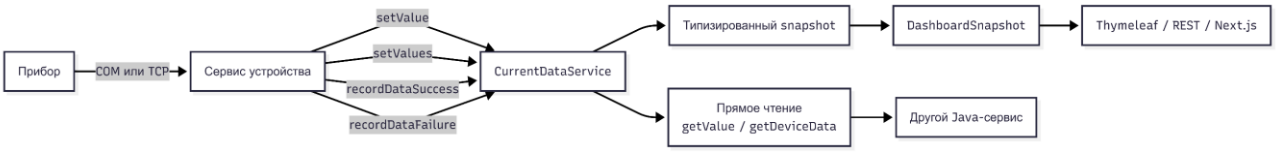
Результат:

data_available = false

last_error = "Scales response timed out"

last_data = предыдущее успешное значение

11. Схема обращения



Как лучше обращаться

Внутри низкоуровневого сервиса допустимо использовать CurrentDataService напрямую.

Для бизнес-логики и автоматического сценария предпочтительнее типизированные snapshots:

ScalesSnapshot scales = scalesService.snapshot();

NicoleStatusView nicole = nicoleService.statusView();

RoboticManipulatorSnapshot manipulator = manipulatorService.snapshot();

DashboardSnapshot dashboard = dashboardSnapshotService.snapshot();

Например, через агрегированный Dashboard:

DashboardSnapshot dashboard = dashboardSnapshotService.snapshot();

var equipment = dashboard.equipment();

var scales = equipment.scales();

var nicole = equipment.nicole();

var manipulator = equipment.manipulator();

Это безопаснее, чем работа с Map<String, Object>: компилятор проверяет названия полей и их типы.

Для внешней программы напрямую обратиться к CurrentDataService нельзя. Следует использовать:

GET /api/v1/dashboard/snapshot

или специализированный endpoint:

GET /api/v1/manipulator/status

Основная реализация находится в [CurrentDataService.java](C:/Projects/RGSCenter/center/src/main/java/com/rgs/center/service/CurrentDataService.java), а преобразование в общую модель Dashboard — в [EquipmentSnapshotAssembler.java](C:/Projects/RGSCenter/center/src/main/java/com/rgs/center/dashboard/EquipmentSnapshotAssembler.java).